

Jupyter Notebook Execution Report

Name: Omar Yasser Ali

ID: 223101306

Date: May 11, 2026

Cell 1: ■ Markdown

COVID-19 Classification using Machine Learning Models

This notebook implements classification models including Logistic Regression, SVM, and Decision Tree for COVID-19 detection using chest X-ray images.

Cell 2: ■ Code

```
import os
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import cv2
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
```

Cell 3: ■ Code

```
from sklearn.linear_model import LinearRegression
```

Cell 4: ■ Code

```
# Load and preprocess data
```

```

def load_data(folder):
    data = []
    labels = []
    for label, class_name in enumerate(['Covid', 'Normal', 'Viral Pneumonia']):
        path = os.path.join(folder, class_name)
        for img_name in os.listdir(path):
            img_path = os.path.join(path, img_name)
            img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
            if img is not None:
                img = cv2.resize(img, (64, 64))
                data.append(img.flatten())
                labels.append(label)
    return np.array(data), np.array(labels)

X_train, y_train = load_data('Covid19-dataset/train')
X_test, y_test = load_data('Covid19-dataset/test')

# Normalize data
X_train = X_train / 255.0
X_test = X_test / 255.0

```

Cell 5: ■ Code

```

# Train and optimize models with hyperparameter tuning
models = {
    'Logistic Regression': {
        'model': LogisticRegression(max_iter=1000),
        'params': {'C': [0.1, 1, 10]}
    },
    'SVM': {
        'model': SVC(),
        'params': {'C': [0.1, 1, 10], 'kernel': ['linear', 'rbf']}
    },
    'Decision Tree': {
        'model': DecisionTreeClassifier(),
        'params': {'max_depth': [None, 10, 20], 'min_samples_split': [2, 5, 10]}
    }
}

```

```

},
'Linear Regression': {
'model': LinearRegression(),
'params': {} # No params for Linear Regression
}
}

best_models = {}

for name, config in models.items():
print(f'Tuning {name}...')
if config['params']:
grid = GridSearchCV(config['model'], config['params'], cv=3, scoring='accuracy')
grid.fit(X_train, y_train)
best_model = grid.best_estimator_
print(f'Best params for {name}: {grid.best_params_}')
else:
best_model = config['model']
best_model.fit(X_train, y_train)

best_models[name] = best_model

# Predict
if name == 'Linear Regression':
y_pred = np.round(best_model.predict(X_test)).astype(int)
y_pred = np.clip(y_pred, 0, 2)
else:
y_pred = best_model.predict(X_test)

print(f'{name} Accuracy: {accuracy_score(y_test, y_pred)}')
print(classification_report(y_test, y_pred))

# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Covid', 'Normal',
'Viral Pneumonia'], yticklabels=['Covid', 'Normal', 'Viral Pneumonia'])
plt.title(f'Confusion Matrix for {name}')

```

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

Output:

Tuning Logistic Regression...

Best params for Logistic Regression: {'C': 0.1}

Logistic Regression Accuracy: 0.8787878787878788

	precision	recall	f1-score	support
0	0.93	0.96	0.94	26
1	0.89	0.80	0.84	20
2	0.81	0.85	0.83	20
accuracy			0.88	66
macro avg	0.87	0.87	0.87	66
weighted avg	0.88	0.88	0.88	66

Tuning SVM...

Best params for SVM: {'C': 0.1, 'kernel': 'linear'}

SVM Accuracy: 0.8636363636363636

	precision	recall	f1-score	support
0	0.92	0.92	0.92	26
1	0.89	0.80	0.84	20
2	0.77	0.85	0.81	20
accuracy			0.86	66
macro avg	0.86	0.86	0.86	66
weighted avg	0.87	0.86	0.86	66

Tuning Decision Tree...

Best params for Decision Tree: {'max_depth': None, 'min_samples_split': 10}

Decision Tree Accuracy: 0.7424242424242424

	precision	recall	f1-score	support
0	1.00	0.88	0.94	26
1	0.69	0.55	0.61	20
2	0.56	0.75	0.64	20
accuracy			0.74	66

macro avg	0.75	0.73	0.73	66
weighted avg	0.77	0.74	0.75	66

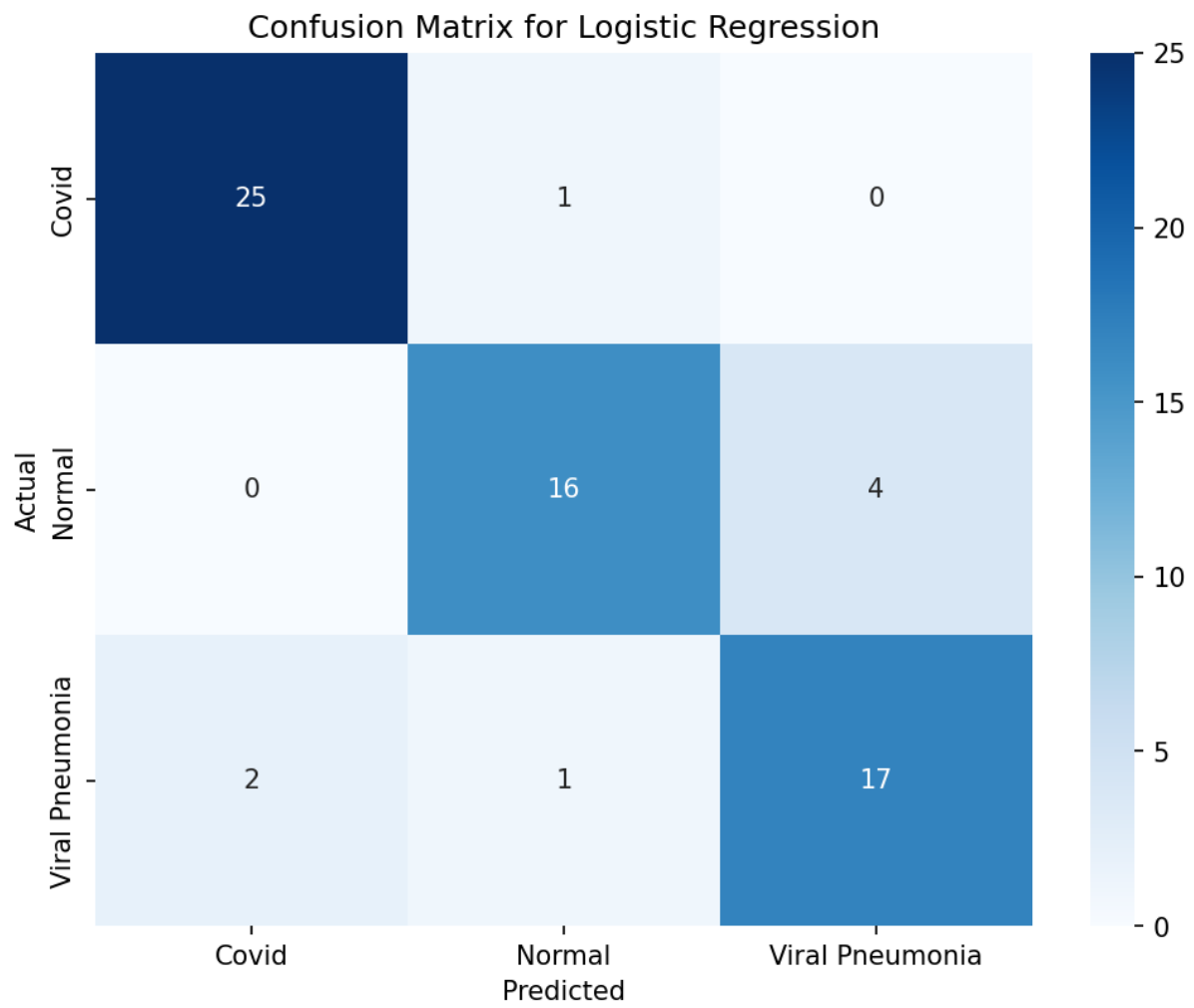
Tuning Linear Regression...

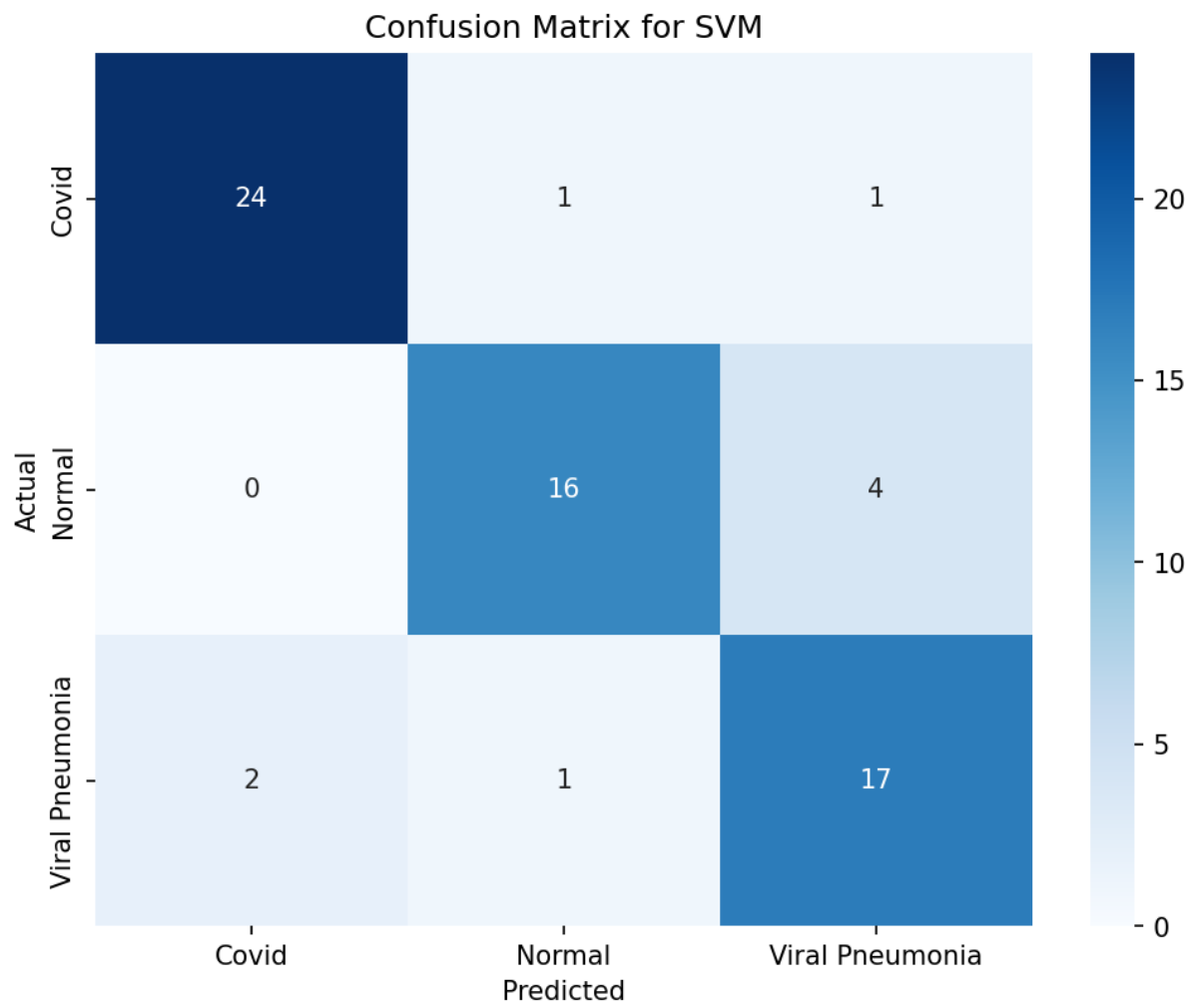
Linear Regression Accuracy: 0.7878787878787878

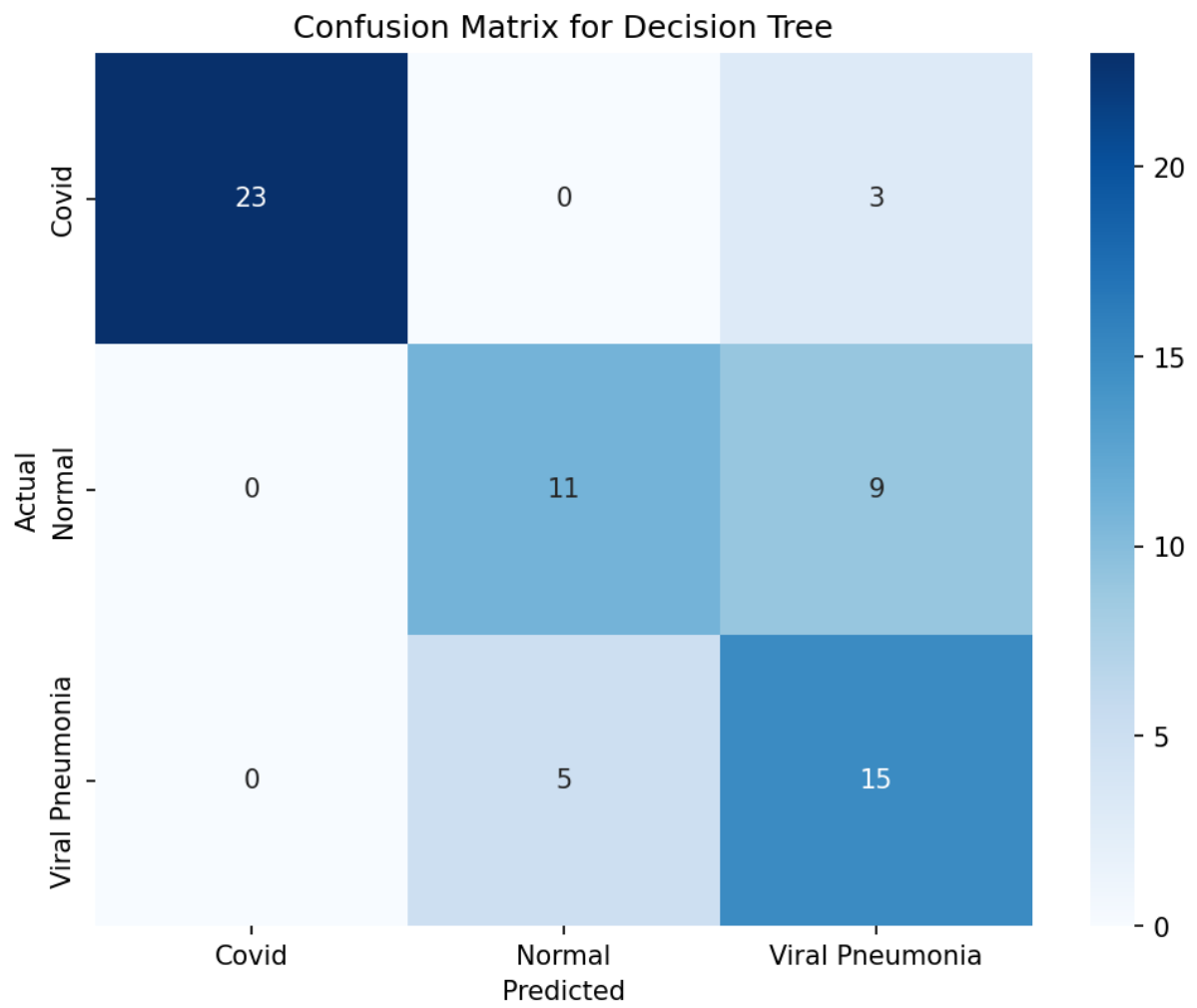
	precision	recall	f1-score	support
0	1.00	0.88	0.94	26
1	0.64	0.70	0.67	20
2	0.71	0.75	0.73	20
accuracy			0.79	66
macro avg	0.78	0.78	0.78	66
weighted avg	0.80	0.79	0.79	66

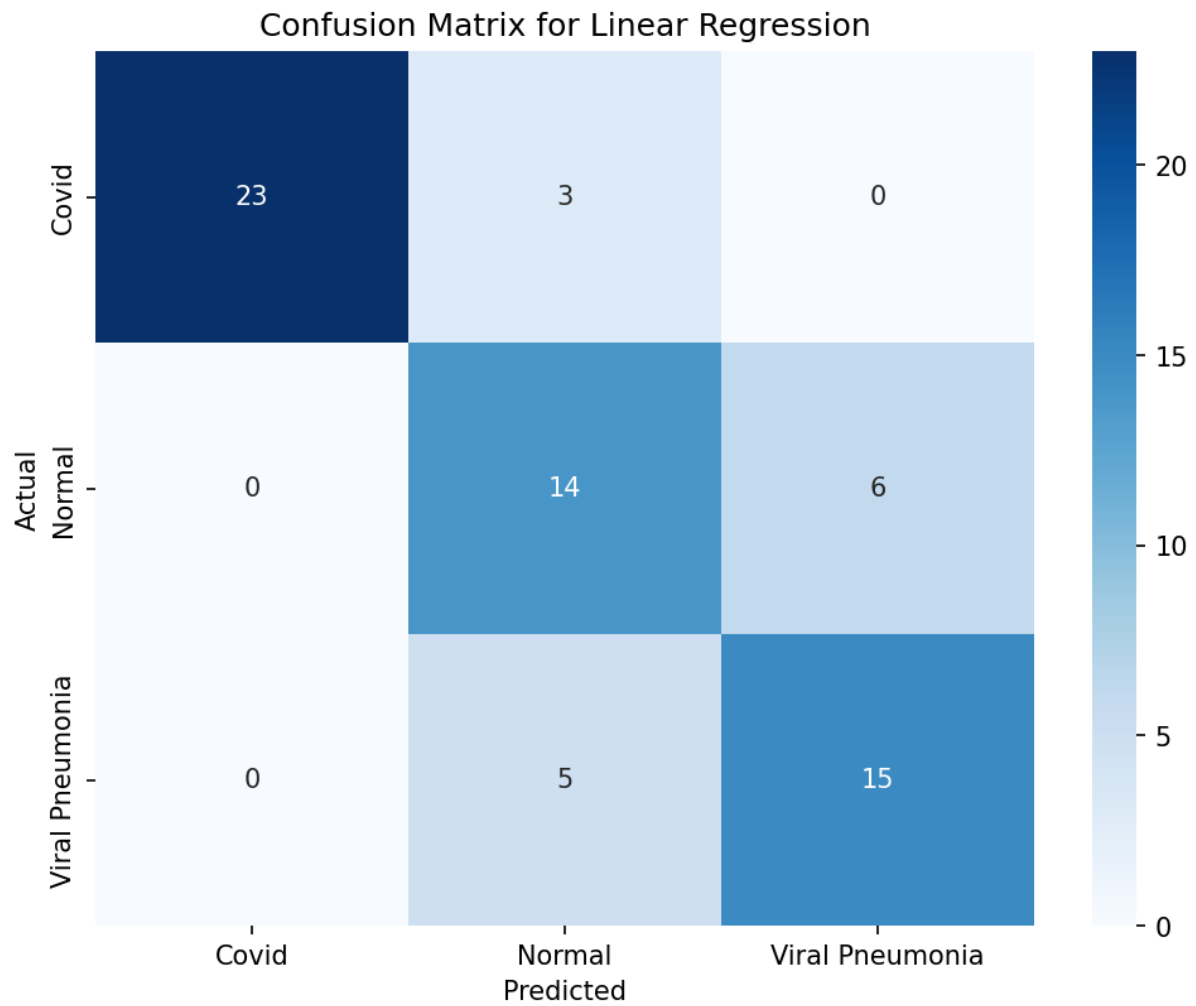
[STDERR]

<string>:1: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown









Cell 6: ■ Code

```
# Export all models
for name, model in best_models.items():
    joblib.dump(model, f'{name.replace(" ", "_").lower()}_model.pkl')
    print(f'Saved {name} model as {name.replace(" ", "_").lower()}_model.pkl')
```

Output:

```
Saved Linear Regression model as linear_regression_model.pkl
```